



Chapter 6. Testing with pytest

If you think back to when you wrote your first programs, you may recall a common experience: you had an idea for how a program could help with a real-life task and spent a sizable amount of time coding it from top to bottom, only to be confronted with screens full of disheartening error messages when you finally ran it. Or, worse, it gave you results that were subtly wrong.

There are a few lessons we've all learned from experiences like this. One is to start simple and keep it simple as you iterate on the program. Another lesson is to test early and repeatedly. Initially, this may just mean to run the program manually and validate that it does what it should. Later on, if you break the program into smaller parts, you can test those parts in isolation and automatically. As a side effect, the program gets easier to read and work on, too.

In this chapter, I'll talk about how testing can help you produce value early and consistently. Good tests amount to an executable specification of the code you own. They set you free from institutional knowledge in a team or company, and they speed up your development by giving you immediate feedback on changes.

The third-party testing framework [pytest](#) has become somewhat of a de facto standard in the Python world. Tests written with pytest are simple and readable: you write most tests as if there was no framework, using basic language primitives like functions and assertions. Despite its simplicity, the framework is powerful and expressive, notably through its concepts of test fixtures and parameterized tests. Pytest is extensible and comes with a rich ecosystem of plugins.

NOTE

Pytest originated in the PyPy project, a Python interpreter written in Python. Early on, the PyPy developers worked on a separate standard library called `std`, later renamed to `py`. Its testing module `py.test` became an independent project under the name `pytest`.

Writing a Test

[Example 6-1](#) revisits the Wikipedia example from [Chapter 3](#). The program is as simple as it gets—yet it’s far from obvious how you’d write tests for it. The `main` function has no inputs and no outputs—only side effects, such as writing to the standard output stream. How would you test a function like this?

Example 6-1. The `main` function from `random-wikipedia-article`

```
def main():
    with urllib.request.urlopen(API_URL) as response:
        data = json.load(response)

    print(data["title"], end="\n\n")
    print(textwrap.fill(data["extract"]))
```

Let’s write an *end-to-end* test that runs the program in a subprocess and checks that it completes with nonempty output. End-to-end tests run the entire program the way an end user would ([Example 6-2](#)).

Example 6-2. A test for `random-wikipedia-article`

```
import subprocess
import sys

def test_output():
    args = [sys.executable, "-m", "random_wikipedia_article"]
    process = subprocess.run(args, capture_output=True, check=True)
    assert process.stdout
```

TIP

Tests written using pytest are functions whose names start with `test`. Use the built-in `assert` statement to check for expected behavior. Pytest rewrites the language construct to provide rich error reporting in case of a test failure.

Place the contents of [Example 6-2](#) in the file `test_main.py` in a `tests` directory. Include an empty `__init__.py` file to turn the test suite into an import package. This lets you mirror the layout of the package you're testing,¹ and it gives you the option to import modules with test utilities.

At this point, your project should be structured in the following way:

```
random-wikipedia-article
├── pyproject.toml
├── src
│   └── random_wikipedia_article
│       ├── __init__.py
│       └── __main__.py
└── tests
    ├── __init__.py
    └── test_main.py
```

Managing Test Dependencies

Tests must be able to import your project and its dependencies, so you need to install pytest in your project environment. For example, add a `tests` extra to your project:

```
[project.optional-dependencies]
tests = ["pytest>=8.1.1"]
```

You can now install pytest in the project environment:

```
$ uv pip install -e ".[tests]"
```

Alternatively, compile a requirements file and synchronize your environment:

```
$ uv pip compile --extra=tests pyproject.toml -o dev-requirements.txt
$ uv pip sync dev-requirements.txt
$ uv pip install -e . --no-deps
```

If you use Poetry, add pytest to your project using `poetry add` instead:

```
$ poetry add --group=tests "pytest>=8.1.1"
```

Please refer to these steps when I ask you to add test dependencies later in this chapter.

Finally, let's run the test suite. If you're on Windows, activate the environment before you run the following command:

```
$ py -m pytest
===== test session starts =====
platform darwin -- Python 3.12.2, pytest-8.1.1, pluggy-1.4.0
rootdir: ...
collected 1 item

tests/test_main.py . [100%]
===== 1 passed in 0.01s =====
```

TIP

Use `py -m pytest` even in Poetry projects. It's both shorter and safer than `poetry run pytest`. If you forget to install pytest into the environment, Poetry falls back to your global environment. (The safe variant would be `poetry run python -m pytest`.)

Designing for Testability

Writing finer-grained tests for the program is much harder. The API endpoint returns a random article, so *which* title and summary should the tests expect? Every invocation sends an HTTP request to the real Wikipedia API. Those network roundtrips will make the test suite excruciatingly slow—and you can run tests only when your machine is connected to the internet.

Python programmers have an arsenal of tools at their disposal for situations like this. Most of these involve some form of *monkey patching*, which replaces functions or objects at runtime to make the code easier to test. For example, you can capture program output by replacing `sys.stdout` with a file-like object that writes to an internal buffer for later inspection. You can replace `urlopen` with a function that returns canned HTTP responses of your liking. Libraries like `responses`, `respx`, or `vcr.py` provide high-level interfaces that monkey patch the HTTP machinery behind the scenes. More generic approaches use the standard `unittest.mock` module or pytest’s `monkeypatch` fixture.

NOTE

The term *monkey patch* for replacing code at runtime originated at Zope Corporation. Initially, people at Zope called the technique “guerilla patching,” since it didn’t abide by the usual rules of patch submission. People heard that as “gorilla patch”—and the more refined versions soon became known as “monkey patches.”

While these tools serve their purpose, I’d encourage you to focus on the root of the problem: [Example 6-1](#) has no separation of concerns. A single function serves as the application entry point, communicates with an external API, and presents the results on the console. This makes it hard to test its features in isolation.

The program also lacks abstraction—in two ways. First, it doesn’t encapsulate implementation details when interfacing with other systems, like

interacting with the Wikipedia API or writing to the terminal. Second, its central concept—the Wikipedia article—appears only as an amorphous JSON object: the program doesn’t abstract its domain model in any way, such as by defining an `Article` class.

[Example 6-3](#) shows a refactoring that makes the code more testable.

While this version of the program is longer, it expresses its logic more clearly and is more amenable to change. Good tests don’t just catch bugs: they improve the design of your code.

Example 6-3. Refactoring for testability

```
import sys ❶
from dataclasses import dataclass

@dataclass
class Article:
    title: str = ""
    summary: str = ""

def fetch(url):
    with urllib.request.urlopen(url) as response:
        data = json.load(response)
    return Article(data["title"], data["extract"])

def show(article, file):
    summary = textwrap.fill(article.summary)
    file.write(f"{article.title}\n\n{summary}\n")

def main():
    article = fetch(API_URL)
    show(article, sys.stdout)
```

- ❶ For brevity, examples in this chapter show imports on first use only.

The refactoring extracts `fetch` and `show` functions from `main`. It also defines an `Article` class as the common denominator of these functions. Let’s see how these changes let you test the parts of the program in isolation and in a repeatable way.

The `show` function accepts any file-like object. While `main` passes `sys.stdout`, tests can pass an `io.StringIO` instance to store the output in memory. [Example 6-4](#) uses this technique to check that the output ends with a newline. The final newline ensures the output doesn't run into the next shell prompt.

Example 6-4. Testing the `show` function

```
import io
from random_wikipedia_article import Article, show

def test_final_newline():
    article = Article("Lorem Ipsum", "Lorem ipsum dolor sit amet.")
    file = io.StringIO()
    show(article, file)
    assert file.getvalue().endswith("\n")
```

There's another benefit to the refactoring: the functions hide the implementation behind an interface that involves only your problem domain—URLs, articles, files. This means your tests are less likely to break when you swap out your implementation. Go ahead and change the `show` function to use `Rich`, as shown in [Example 6-5](#).² You won't need to adapt your tests!

Example 6-5. Swapping out the implementation of `show`

```
from rich.console import Console

def show(article, file):
    console = Console(file=file, width=72, highlight=False)
    console.print(article.title, style="bold", end="\n\n")
    console.print(article.summary)
```

In fact, the whole point of tests is to give you confidence that your program still works after making changes like this. Mocks and monkey patches, on the other hand, are brittle: they tie your test suite to implementation details, making it increasingly hard to change your program down the road.

Fixtures and Parameterization

Here are some other properties of the `show` function that you might check for:

- It should include all the words of the title and summary.
- There should be a blank line after the title.
- The summary should not exceed a line length of 72 characters.

Every test for the `show` function starts by setting up an output buffer. You can use a fixture to remove this code duplication. *Fixtures* are functions declared with the `pytest.fixture` decorator:

```
@pytest.fixture
def file():
    return io.StringIO()
```

Tests (and fixtures) can use a fixture by including a function parameter with the same name. When pytest invokes the test function, it passes the return value of the fixture function. Let's rewrite [Example 6-4](#) to use the fixture:

```
def test_final_newline(file):
    article = Article("Lorem Ipsum", "Lorem ipsum dolor sit amet.")
    show(article, file)
    assert file.getvalue().endswith("\n")
```

WARNING

If you forget to add the parameter `file` to the test function, you get a confusing error: `'function' object has no attribute 'write'`. This happens because the name `file` now refers to the fixture function in the same module.³

If every test used the same article, you'd likely miss some edge cases. For example, you don't want your program to crash if an article comes with

an empty title. [Example 6-6](#) runs the test for a number of articles with the `@pytest.mark.parametrize` decorator.⁴

Example 6-6. Running tests against multiple articles

```
articles = [
    Article(),
    Article("test"),
    Article("Lorem Ipsum", "Lorem ipsum dolor sit amet."),
    Article(
        "Lorem ipsum dolor sit amet, consectetur adipiscing elit",
        "Nulla mattis volutpat sapien, at dapibus ipsum accumsan eu."
    ),
]

@pytest.mark.parametrize("article", articles)
def test_final_newline(article, file):
    show(article, file)
    assert file.getvalue().endswith("\n")
```

If you parameterize many tests in the same way, you can create a *parameterized fixture*, a fixture with multiple values ([Example 6-7](#)). As before, pytest runs the test once for each article in `articles`.

Example 6-7. Parameterized fixture for running tests against multiple articles

```
@pytest.fixture(params=articles)
def article(request):
    return request.param

def test_final_newline(article, file):
    show(article, file)
    assert file.getvalue().endswith("\n")
```

So what did you gain here? For one thing, you don't need to decorate each test with `@pytest.mark.parametrize`. There's another advantage if your tests aren't all in the same module: you can place fixtures in a file named *conftest.py* and use them across your entire test suite without imports.

The syntax for parameterized fixtures is somewhat arcane. To keep things simple, I like to define a small helper:

```
def parametrized_fixture(*params):  
    return pytest.fixture(params=params)(lambda request: request.param)
```

Use the helper to simplify the fixture from [Example 6-7](#). You can also inline the `articles` variable from [Example 6-6](#):

```
article = parametrized_fixture(Article(), Article("test"), ...)
```

THE UNITTEST FRAMEWORK

The standard library includes a testing framework in the `unittest` module—inspired by JUnit, a Java testing library from the early days of test-driven development. Let's rewrite the test using `unittest` and compare it with the `pytest` version:

```
import unittest

class TestShow(unittest.TestCase):
    def setUp(self):
        self.article = Article("Lorem Ipsum", "Lorem ipsum dolor sit amet.")
        self.file = io.StringIO()

    def test_final_newline(self):
        show(self.article, self.file)
        self.assertEqual("\n", self.file.getvalue()[-1])
```

In `unittest`, tests are methods with names that start with `test` in a class derived from `unittest.TestCase`. The `assert*` methods let you check for expected properties. The `setUp` method prepares the *test environment* for each test—the test objects each test uses. In this case, you're setting up an `Article` instance and an output buffer for the `show` function.

Run the test suite from the project directory using the command `py -m unittest`:

```
$ py -m unittest
.
-----
Ran 1 test in 0.000s

OK
```

Writing tests with `unittest` saves you a third-party dependency. If you're already familiar with a JUnit-style framework from another language, you'll feel right at home. But there are several problems with its design:

- The class tightly couples tests and the test environment. As a result, you can't reuse test objects as easily as with pytest fixtures.
 - The framework uses inheritance to provide shared functionality. This couples tests with the framework plumbing, all in a single namespace and instance.
 - Placing tests in a class makes them less readable than a module with functions.
 - The assertion methods lack expressivity and generality—every type of check requires a dedicated method. For example, you have `assertEqual` and `assertIn`, but there's no `assertStartsWith`.
-

TIP

If you have a test suite written with `unittest`, there's no need to rewrite it to start using pytest—pytest “speaks” `unittest`, too. Use pytest as a test runner right away and you can rewrite your test suite incrementally later.

Advanced Techniques for Fixtures

For the `fetch` function, tests can set up a local HTTP server and perform a *roundtrip* check. This is shown in [Example 6-8](#): you serve an `Article` instance via HTTP, fetch the article from the server, and check that the served and fetched instances are equal.

Example 6-8. Testing the `fetch` function (version 1)

```
def test_fetch(article):  
    with serve(article) as url:  
        assert article == fetch(url)
```

The `serve` helper function takes an article and returns a URL for fetching the article. More precisely, it wraps the URL in a *context manager*, an object for use in a `with` block. This allows `serve` to clean up after itself when you exit the `with` block—by shutting down the server:

```

from contextlib import contextmanager

@contextmanager
def serve(article):
    ... # start the server
    yield f"http://localhost:{server.server_port}"
    ... # shut down the server

```

You can implement the `serve` function using the `http.server` module from the standard library ([Example 6-9](#)). Don't worry too much about the details, though. Later in this chapter, I'll introduce the `pytest-httpserver` plugin, which will bear the brunt of the work.

Example 6-9. The `serve` function

```

import http.server
import json
import threading

@contextmanager
def serve(article):
    data = {"title": article.title, "extract": article.summary}
    body = json.dumps(data).encode()

    class Handler(http.server.BaseHTTPRequestHandler): ❶
        def do_GET(self):
            self.send_response(200)
            self.send_header("Content-Type", "application/json")
            self.send_header("Content-Length", str(len(body)))
            self.end_headers()
            self.wfile.write(body)

    with http.server.HTTPServer(("localhost", 0), Handler) as server: ❷
        thread = threading.Thread(target=server.serve_forever, daemon=True) ❸
        thread.start()
        yield f"http://localhost:{server.server_port}"
        server.shutdown()
        thread.join()

```

The request handler responds to every GET request with a UTF-8 encoded JSON representation of the article.

- ❷ The server accepts only local connections. The operating system randomly assigns the port number.
- ❸ The server runs in a background thread. This allows control to return to the tests.

Firing up and shutting down a web server for every test is expensive. Would it help to turn the server into a fixture? At first glance, not much—every test gets its own instance of a fixture. However, you can instruct pytest to create a fixture only once during the entire test session, using a *session-scoped fixture*:

```
@pytest.fixture(scope="session")
def httpserver():
    ...
```

That looks more promising, but how do you shut down the server when the tests are done with it? Up to now, your fixtures have only prepared a test object and returned it. You can't run code after a `return` statement. However, you *can* run code after a `yield` statement—so pytest allows you to define a fixture as a generator.

A *generator fixture* prepares a test object, yields it, and cleans up resources at the end—similar to a context manager. You use it in the same way as an ordinary fixture that returns its test object. Pytest handles the setup and teardown phases behind the scenes and calls your test function with the yielded value.

[Example 6-10](#) defines the `httpserver` fixture using the generator technique.

Example 6-10. The `httpserver` fixture

```
@pytest.fixture(scope="session")
def httpserver():
```

```

class Handler(http.server.BaseHTTPRequestHandler):
    def do_GET(self):
        article = self.server.article ❶
        data = {"title": article.title, "extract": article.summary}
        body = json.dumps(data).encode()
        ... # as before

with http.server.HTTPServer(("localhost", 0), Handler) as server:
    thread = threading.Thread(target=server.serve_forever, daemon=True)
    thread.start()
    yield server
    server.shutdown()
    thread.join()

```

- ❶ Unlike in [Example 6-9](#), there's no `article` in scope. Instead, the request handler accesses it from the `article` attribute on the server (see [Example 6-11](#)).

There's still a missing piece: you need to define the `serve` function. The function now depends on the `httpserver` fixture to do its work, so you can't just define it at the module level. Let's move it into the test function for now ([Example 6-11](#)).

Example 6-11. Testing the `fetch` function (version 2)

```

def test_fetch(article, httpserver):
    def serve(article):
        httpserver.article = article ❶
        return f"http://localhost:{httpserver.server_port}"

    assert article == fetch(serve(article))

```

- ❶ Store the article in the server so the request handler can access it.

The `serve` function no longer returns a context manager, just a plain URL—the `httpserver` fixture handles all of the setting up and tearing down. But you can still do better. The nested function clutters the test—and every other test for the `fetch` function. Instead, let's define `serve`

inside its own fixture—after all, fixtures can return any object, including functions ([Example 6-12](#)).

Example 6-12. The `serve` fixture

```
@pytest.fixture
def serve(httpserver): ❶
    def f(article): ❷
        httpserver.article = article
        return f"http://localhost:{httpserver.server_port}"
    return f
```

- ❶ The outer function defines a `serve` fixture, which depends on `httpserver`.
- ❷ The inner function is the `serve` function you call in your tests.

Thanks to the `serve` fixture, the test function becomes a one-liner ([Example 6-13](#)). It's also much faster, because you start and stop the server only once per session.

Example 6-13. Testing the `fetch` function (version 3)

```
def test_fetch(article, serve):
    assert article == fetch(serve(article))
```

Your test isn't tied to any particular HTTP client library. [Example 6-14](#) swaps out the implementation of the `fetch` function to use HTTPX.⁵ This would have broken any test that used monkey patching—but your test will still pass!

Example 6-14. Swapping out the implementation of `fetch`

```
import httpx

from importlib.metadata import metadata

USER_AGENT = "{Name}/{Version} (Contact: {Author-email})"
```

```
def fetch(url):
    fields = metadata("random-wikipedia-article")
    headers = {"User-Agent": USER_AGENT.format_map(fields)}

    with httpx.Client(headers=headers, http2=True) as client:
        response = client.get(url, follow_redirects=True)
        response.raise_for_status()
        data = response.json()

    return Article(data["title"], data["extract"])
```

Extending pytest with Plugins

As you’ve seen in [“Entry Points”](#), pytest’s extensible design lets anybody contribute pytest plugins and publish them to PyPI,⁶ so a rich ecosystem of plugins has evolved. You’ve already seen the `pytest-sugar` plugin, which enhances pytest’s output, including adding a progress bar. In this section, you’ll look at a few more.

The pytest-httpserver Plugin

The `pytest-httpserver` [plugin](#) provides an `httpserver` fixture that’s more versatile and battle-tested than [Example 6-10](#). Let’s use this plugin.

First, add `pytest-httpserver` to your test dependencies. Next, remove the existing `httpserver` fixture from your test module. Finally, update the `serve` fixture to use the plugin ([Example 6-15](#)).

Example 6-15. The `serve` fixture using `pytest-httpserver`

```
@pytest.fixture
def serve(httpserver):
    def f(article):
        json = {"title": article.title, "extract": article.summary}
        httpserver.expect_request("/").respond_with_json(json)
        return httpserver.url_for("/")
    return f
```

[Example 6-15](#) configures the server to respond to requests to `"/"` with the JSON representation of the article. The plugin offers much flexibility beyond this use case—for example, you can add custom request handlers or communicate over HTTPS.

The pytest-xdist Plugin

As your test suite grows, you'll be looking for ways to speed up test runs. Here's an easy way: utilize all your CPU cores. The `pytest-xdist` [plugin](#) spawns a worker process on each processor and distributes tests randomly across the workers. The randomization also helps detect hidden dependencies between your tests.

Add `pytest-xdist` to your test dependencies and update your environment. Use the option `--numprocesses` or `-n` to specify the number of worker processes. Specify `auto` to use all physical cores on your system:

```
$ py -m pytest -n auto
```

The factory-boy and faker Libraries

In [“Fixtures and Parameterization”](#), you hardcoded the articles your tests run against. Let's avoid this boilerplate—it makes your tests hard to maintain.

Instead, the `factory-boy` [library](#) lets you create factories for test objects. You can generate batches of objects with predictable attributes, such as by using a sequence number. Alternatively, you can populate attributes randomly using the `faker` [library](#).

Add `factory-boy` to your test dependencies and update your environment. [Example 6-16](#) defines a factory for randomized articles and creates a batch of ten articles for the `article` fixture. (If you want to see `pytest-xdist` in action, increase the number of articles and run pytest with `-n auto`.)

Example 6-16. Creating a batch of articles with `factory-boy` and `faker`

```
from factory import Factory, Faker

class ArticleFactory(Factory):
    class Meta:
        model = Article ❶

        title = Faker("sentence") ❷
        summary = Faker("paragraph")

article = parametrized_fixture(*ArticleFactory.build_batch(10)) ❸
```

- ❶ Specify the class of the test objects with `Meta.model`.
- ❷ Use a random sentence for the title and a random paragraph for the summary.
- ❸ Generate a batch of articles using the `build_batch` method.

This simplified factory doesn't cover edge cases particularly well. For a real-world application, you should include empty and extremely large strings, as well as unusual characters such as control characters. Another great testing library that lets you explore the search space of possible inputs is `hypothesis`.

Other Plugins

Pytest plugins perform a variety of functions ([Table 6-1](#)). These include executing tests in parallel or in random order, presenting or reporting test results in custom ways, and integrating with frameworks and other tools. Many plugins provide useful fixtures—such as for interacting with external systems or creating test doubles.⁷

Table 6-1. A selection of pytest plugins

Plugin	Category	Description	Option
<code>pytest-xdist</code>	Execution	Distribute tests across multiple CPUs	<code>--numprocesses</code>
<code>pytest-sugar</code>	Presentation	Enhance output with a progress bar	
<code>pytest-icdiff</code>	Presentation	Show colorized diffs on test failures	
<code>anyio</code>	Frameworks	Use asynchronous tests with asyncio and trio	
<code>pytest-httpserver</code>	Fake servers	Spawn an HTTP server with canned responses	
<code>pytest-factoryboy</code>	Fake data	Turn factories into fixtures	
<code>pytest-datadir</code>	Storage	Access static data in your test suite	
<code>pytest-cov</code>	Coverage	Produce coverage reports with Coverage.py	<code>--cov</code>
<code>xdoctest</code>	Documentation	Run code examples from docstrings	<code>--xdoctest</code>
<code>typeguard</code>	Type checking	Type-check your code at runtime	<code>--typeguard-packages</code>

TIP

Find each project on PyPI at <https://pypi.org/project/<name>>. The project homepage is available under *Project links* in the navigation bar.

Summary

In this chapter, you’ve learned how to test your Python projects with pytest:

- Tests are functions that exercise your code and check for expected behavior using the `assert` built-in. Prefix their names—and the names of the containing modules—with `test_`, and pytest will discover them automatically.
- Fixtures are functions or generators that set up and tear down test objects; declare them with the `@pytest.fixture` decorator. You can use a fixture in a test by including a parameter named like the fixture.
- Plugins for pytest can provide useful fixtures, as well as modify test execution, enhance reporting, and much more.

One of the prime characteristics of good software is that it’s easy to change, since any piece of code used in the real world must adapt to evolving requirements and an ever-changing environment. Tests make change easier in several ways:

- They drive software design toward loosely coupled building blocks that you can test in isolation: fewer interdependencies mean fewer barriers to change.
- They document and enforce expected behavior. That gives you the freedom and confidence to continuously refactor your codebase—keeping it maintainable as it grows and transforms.
- They reduce the cost of change by detecting defects early. The earlier you catch an issue, the cheaper it is to work out the root cause and develop a fix.

End-to-end tests give you high confidence that a feature works as designed—but they’re slow, flaky, and bad at isolating the causes of failures. Let most of your tests be unit tests. Avoid monkey patching to break the dependencies of your code for testability. Instead, decouple the core of your application from I/O, external systems, and third-party frameworks. Good software design and a solid testing strategy will keep your test suite blazingly fast and resilient to change.

This chapter focuses on the tooling side of things, but there's so much more to good testing practices. Luckily, other people have written fantastic texts about this topic. Here are three of my all-time favorites:

- Kent Beck, *Test-Driven Development: By Example* (London: Pearson, 2002).
- Michael Feathers, *Working Effectively with Legacy Code* (London: Pearson, 2004).
- Harry Percival and Bob Gregory, [*Architecture Patterns in Python*](#) (Sebastopol: O'Reilly, 2020).

If you want to know all about how to test with pytest, read Brian's book:

- Brian Okken, *Python Testing with pytest: Simple, Rapid, Effective, and Scalable*, Second Edition (Raleigh: The Pragmatic Bookshelf, 2022).

- 1** Large packages can have modules with the same name—say, `gizmo.foo.registry` and `gizmo.bar.registry`. Under pytest's default [import mode](#), test modules must have unique fully qualified names—so you must place the `test_registry` modules in separate `tests.foo` and `tests.bar` packages.
- 2** Remember to add Rich to your project as described in [“Specifying Dependencies for a Project”](#). If you use Poetry, refer to [“Managing Dependencies”](#).
- 3** My reviewer Hynek recommends a technique to avoid this pitfall and get an idiomatic `NameError` instead. The trick is to name the fixture explicitly with `@pytest.fixture(name="file")`. This lets you use a private name for the function, such as `_file`, that doesn't collide with the parameter.
- 4** Note the somewhat uncommon spelling variant *parametrize* instead of *parameterize*.
- 5** Remember to add a dependency on `httpx[http2]` to your project.
- 6** The [cookiecutter-pytest-plugin template](#) gives you a solid project structure for writing your own plugin.
- 7** *Test double* is the umbrella term for the various kinds of objects tests use in lieu of the real objects used in production code. A good overview is [“Mocks Aren't Stubs”](#)

